MANIPAL ACADEMY OF HIGHER EDUCATION

(Institution of Eminence Deemed to be University)

MANIPAL INSTITUTE OF TECHNOLOGY

School of Computer Engineering

CES 2112 Digital Systems Lab

(Open Ended Experiment Report)

DESIGN AND VERIFICATION OF A 32-BIT SINGLE-CYCLE RISC-V PROCESSOR CORE

Submitted by

Name	Section	Registration No.	Roll No.
Kuldeep Bhurani	CSE-K	251090052182	K054
Saksham Kanodia	CSE-K	251090052618	K062
Yashasvi Sharma	CSE-K	251090052336	K056
Amarnath K.	CSE-K	251090052394	K057

Under the guidance of

Dr. Sheena K. S.
Assistant Professor
School of Computer Engineering, MIT Manipal

Date: 21/10/2026

Contents

1 Problem Statement **1**
- 1.1 System Requirements . . . 1

2 Objectives **1**
- 2.1 Primary Objective . . . 1
- 2.2 Specific Objectives . . . 1
- 2.3 Scope and Deliverables . . . 1

3 System Design and Block Diagram **2**
- 3.1 Proposed Architecture . . . 2
- 3.2 Module Description . . . 2
- 3.3 Supported Instruction Subset . . . 3
- 3.4 Design Decisions and Boundaries . . . 3

4 Verilog HDL Implementation **4**
- 4.1 Top-Level Datapath . . . 4
- 4.2 Control Unit . . . 5
- 4.3 Register File . . . 7
- 4.4 Arithmetic Logic Unit . . . 7
- 4.5 Testbench Design . . . 8

5 Simulation, Verification and Results **8**
- 5.1 Experimental Setup . . . 8
- 5.2 Module-Level Verification . . . 9
- 5.3 Core-Level Test Cases . . . 9
- 5.4 Simulation Waveform . . . 10
- 5.5 Results and Discussion . . . 10

6 Individual Contribution **11**
- 6.1 Contribution Statement . . . 11

7 Conclusion and Future Scope **11**
- 7.1 Conclusion . . . 11
- 7.2 SDG 9: Industry, Innovation and Infrastructure . . . 12
- 7.3 Limitations and Future Scope . . . 12

References **12**

1 PROBLEM STATEMENT

This project implements a compact, single-cycle processor core in Verilog HDL. The RTL uses 32-bit datapath signals and selected instruction encodings from the RISC-V instruction-set architecture. Its main blocks are the program counter (PC), instruction memory, instruction decoder/control unit, register file, immediate generator, arithmetic logic unit (ALU), data memory, and write-back path [1, 2].

The supported instruction subset comprises load word (`lw`), store word (`sw`), branch if equal (`beq`), and R-type `add`, `and`, and `or`. The register file contains 32 32-bit registers, with x0 returning zero and ignoring writes. Instruction and data memories each contain 64 words (256 bytes), and both use byte addresses with the two lowest address bits omitted from word indexing.

1.1 System Requirements

Table I summarizes the core requirements and their verification methods.

TABLE I. SYSTEM REQUIREMENTS AND VERIFICATION METHODS

ID	Requirement	Verification method
R1	Implement the fetch, decode, execute, memory, write-back, and PC-update datapath.	Full-core regression tests
R2	Support `lw`, `sw`, `beq`, `add`, `and`, and `or`.	Instruction-level core tests
R3	Ensure x0 reads as zero and ignores writes.	Register-file and core tests
R4	Generate I-, S-, and B-type sign-extended immediates.	Immediate vectors and BEQ integration test
R5	Verify ADD, SUB, AND, OR, and ALU zero-flag behaviour.	ALU testbench
R6	Verify asynchronous memory reads and synchronous data-memory writes.	Memory and store testbenches

2 OBJECTIVES

2.1 Primary Objective

To design and verify a 32-bit, single-cycle RISC-V processor core for a defined subset of instructions using Verilog HDL.

2.2 Specific Objectives

1. Integrate the datapath and control logic for the supported instructions.
2. Implement a 32-register, 32-bit register file with two asynchronous read ports and one synchronous write port.
3. Implement instruction and data memories that can be initialized from hexadecimal files.
4. Implement I-, S-, and B-type immediate extraction and sign extension.
5. Implement ALU addition, subtraction, bitwise AND, and bitwise OR with a zero flag.
6. Verify supported operations using module-level and full-core tests.

2.3 Scope and Deliverables

Deliverables comprise the synthesizable Verilog modules, ten passing testbench targets, hexadecimal test programs/data, build configuration, simulation traces, and this report. The testbenches generate VCD files for waveform inspection, and the BEQ timing trace is verified in GTKWave.

3 SYSTEM DESIGN AND BLOCK DIAGRAM

3.1 Proposed Architecture

The processor fetches a 32-bit instruction using the current PC. The instruction decoder extracts the opcode, `funct3`, `funct7`, destination register, and source-register indices, then generates control signals. The register file supplies two source operands. The immediate generator reconstructs an I-, S-, or B-type immediate when required. A mux selects the ALU's second operand from either the second register read or the immediate.

The ALU computes arithmetic/logical results and asserts its zero output when the result is zero. For `lw` and `sw`, its result is the effective byte address. The second register-file read supplies the store data. A write-back mux selects data-memory output for `lw` and the ALU result for the supported R-type instructions. For `beq`, the ALU subtracts the two register operands; when the zero flag is asserted, the B-type immediate is added to the current PC. Otherwise, the PC advances by four bytes. This separation of datapath and control follows common processor-organization models [3, 4].

Figure 1 shows the principal functional blocks and the main datapath connections.

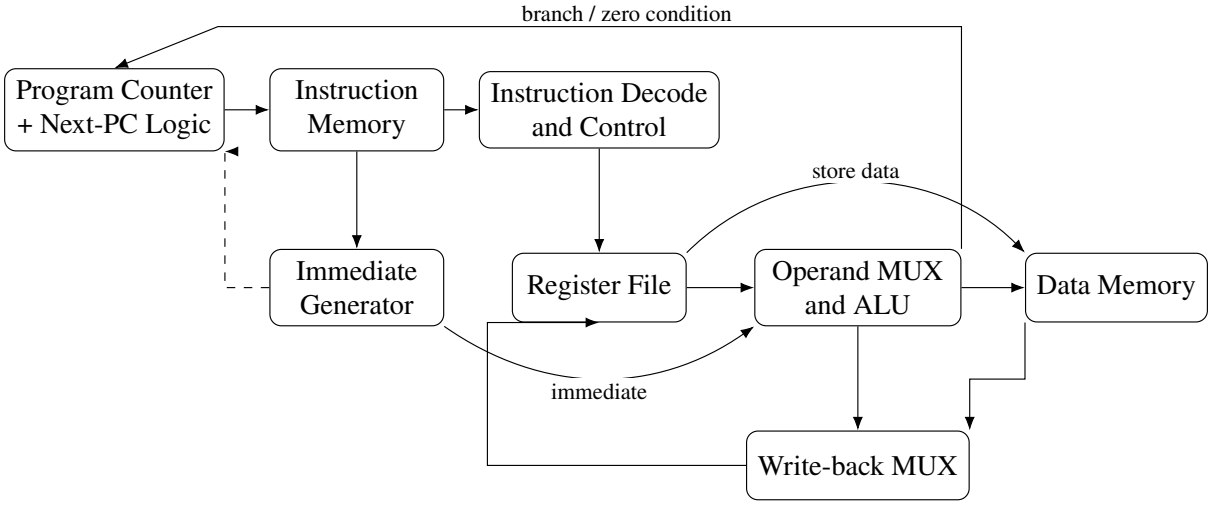


Fig. 1. High-level datapath and control blocks of the implemented processor.

3.2 Module Description

Table II lists the major source modules in the supplied project archive [2].

TABLE II. MAJOR RTL MODULES

Module	Main inputs	Purpose / outputs
`main`	`clk`, `rst`, memory-file parameters	Integrates the PC, datapath, control, and memories
`control`	Opcode, `funct3`, `funct7`	Generates register, memory, immediate, ALU, and branch control signals
`regfile`	Clock/reset, register indices, write data/enable	32×32-bit register file with two asynchronous reads
`instruction_memory`	Byte address, optional initialization file	32-bit instruction output; 64 words by default
`data_memory`	Clock, byte address, write data/enable, optional file	Asynchronous read and synchronous write RAM
`signext`	Instruction fields, `imm_src`	Reconstructs I-, S-, and B-type immediates
`alu`	Two operands, ALU control	32-bit result and zero flag

3.3 Supported Instruction Subset

The supported encodings and ALU operations are summarized in Table III. The encodings follow the RISC-V instruction formats and assembly conventions; the implementation supports only the operations listed here [1, 2, 5].

TABLE III. SUPPORTED INSTRUCTION SUBSET

Instruction	Opcode	`funct3`	`funct7`	Operation
`lw`	`0000011`	`010`	–	Add base register and I-type immediate; write memory data to `rd`
`sw`	`0100011`	`010`	–	Add base register and S-type immediate; write `rs2` to memory
`add`	`0110011`	`000`	`0000000`	Add `rs1` and `rs2`
`and`	`0110011`	`111`	`0000000`	Bitwise AND
`or`	`0110011`	`110`	`0000000`	Bitwise OR
`beq`	`1100011`	`000`	–	Subtract operands; branch if the result is zero

3.4 Design Decisions and Boundaries

The PC and register-file writes update on the rising clock edge. The register file has two asynchronous read ports and one synchronous write port. Data memory has an asynchronous read port and a synchronous write port. Instruction and data memories are initialized to zero and then optionally populated from hexadecimal files. Both memories contain 64 32-bit words by default.

Memory addresses are byte addresses, but indices are selected using address bits `[7:2]` for the default six-bit word index. Thus, the two lowest address bits are ignored. The implementation assumes word-aligned accesses; it does not generate a misalignment exception.

4 VERILOG HDL IMPLEMENTATION

4.1 Top-Level Datapath

The `main` module connects the PC, instruction memory, decoder, register file, immediate generator, ALU, data memory, and write-back mux. Listing 1 presents the complete top-level RTL, including instruction fetch and decode, PC update, branch selection, memory access, and write-back [2, 6, 7].

Listing 1: Top-level processor integration (main.v)

```
`ifndef MAIN_V
`define MAIN_V
// ------------------------------------------------------------
//
//          +----+   +--------+   +---------+   +-----+   +--------+
//   PC --->|imem|-->| decode |-->| regfile |-->| ALU |-->|  dmem  |--+
//    ^     +----+   | control|   +---------+   +-----+   +--------+  |
//    |              +--------+       ^  signext--^                    |
//    +-- pc+4                        +------ result mux <-------------+
//
// ------------------------------------------------------------
`include "regfile.v"
`include "instruction_memory.v"
`include "data_memory.v"
`include "signext.v"
`include "alu.v"
`include "control.v"

`timescale 1ns/1ps

module main #(
    parameter IMEM_FILE = "",
    parameter DMEM_FILE = ""
) (
    input wire clk,
    input wire rst
);

    // program counter
    reg  [31:0] pc;
    wire [31:0] pc_plus4 = pc + 32'd4;

    // fetch
    wire [31:0] instruction;

    instruction_memory #(.INIT_FILE(IMEM_FILE)) imem (
        .address     (pc),
        .instruction (instruction)
    );

    // decode
    wire [6:0] op     = instruction[6:0];
    wire [6:0] funct7 = instruction[31:25];
    wire [4:0] rd     = instruction[11:7];
    wire [2:0] funct3 = instruction[14:12];
    wire [4:0] rs1    = instruction[19:15];
    wire [4:0] rs2    = instruction[24:20];

    wire       reg_write, alu_src, mem_write, result_src, branch;
    wire [1:0] imm_src;
    wire [2:0] alu_control;

    control ctrl (
        .op          (op),
        .funct3      (funct3),
        .funct7      (funct7),
        .reg_write   (reg_write),
        .imm_src     (imm_src),
        .alu_src     (alu_src),
        .mem_write   (mem_write),
        .result_src  (result_src),
        .alu_control (alu_control),
        .branch      (branch)
    );
```

```
65
66      wire [31:0] read_data1, read_data2, immediate;
67      wire [31:0] result;               // value written back to rd
68
69      regfile regfile_inst (
70          .clk          (clk),
71          .rst          (rst),
72          .address1     (rs1),
73          .address2     (rs2),
74          .address3     (rd),
75          .write_enable (reg_write),
76          .write_data   (result),
77          .read_data1   (read_data1),
78          .read_data2   (read_data2)
79      );
80
81      signext signext_inst (
82          .raw_src   (instruction[31:7]),
83          .imm_src   (imm_src),
84          .immediate (immediate)
85      );
86
87      // execute
88      wire [31:0] alu_src2 = alu_src ? immediate : read_data2;
89      wire [31:0] alu_result;
90      wire        zero;
91
92      alu alu_inst (
93          .src1        (read_data1),
94          .src2        (alu_src2),
95          .alu_control (alu_control),
96          .alu_result  (alu_result),
97          .zero        (zero)
98      );
99
100     wire [31:0] pc_next = (branch && zero) ? pc + immediate : pc_plus4;
101
102     always @(posedge clk) begin
103         if (rst) pc <= 32'b0;
104         else     pc <= pc_next;
105     end
106
107     // memory / write-back
108     wire [31:0] mem_read_data;
109
110     data_memory #(.INIT_FILE(DMEM_FILE)) dmem (
111         .clk          (clk),
112         .address      (alu_result),
113         .write_data   (read_data2),
114         .write_enable (mem_write),
115         .read_data    (mem_read_data)
116     );
117
118     assign result = result_src ? mem_read_data : alu_result;
119
120 endmodule
121 `endif
```

The branch target is calculated relative to the current PC, not PC+4. The next-PC equation is

$$PC_{\text{next}} = \begin{cases} PC + Imm_B, & \text{if } branch = 1 \text{ and } zero = 1, \\ PC + 4, & \text{otherwise.} \end{cases} \tag{1}$$

4.2 Control Unit

The control unit recognizes the supported load, store, branch, and R-type opcode families. Loads select the I-type immediate and memory-data write-back; stores select the S-type immediate and enable memory writes; BEQ selects the B-type immediate and SUB comparison. For R-type instructions, `funct3` selects ADD, AND, or OR as shown in Table IV. Listing 2 provides the complete decoder.

TABLE IV. ALU CONTROL ENCODINGS

ALU control	Operation
000	ADD
001	SUB
010	AND
011	OR

Listing 2 gives the decoder implementation used to generate the datapath control signals.

Listing 2: Control-unit decoder (control.v)

```
`ifndef CONTROL_V
`define CONTROL_V
`timescale 1ns/1ps

module control (
    input  wire [6:0] op,
    input  wire [2:0] funct3,
    input  wire [6:0] funct7,
    output reg        reg_write,
    output reg  [1:0] imm_src,
    output reg        alu_src,        // 0: rs2, 1: immediate
    output reg        mem_write,
    output reg        result_src,     // 0: ALU result, 1: data memory
    output reg  [2:0] alu_control,
    output reg        branch
);

    localparam OP_LW = 7'b0000011;
    localparam OP_SW = 7'b0100011;
    localparam OP_R  = 7'b0110011;
    localparam OP_B  = 7'b1100011;

    reg [1:0] alu_op;               // 00 : force ADD, 10 : R-type (decode funct3)

    // Main decoder: safe defaults make unsupported instructions no-ops.
    always @(*) begin
        reg_write  = 1'b0;
        imm_src    = 2'b00;
        alu_src    = 1'b0;
        mem_write  = 1'b0;
        result_src = 1'b0;
        branch     = 1'b0;
        alu_op     = 2'b00;

        case (op)
            OP_LW: if (funct3 == 3'b010) begin
                reg_write  = 1'b1;
                imm_src    = 2'b00;   // I-type
                alu_src    = 1'b1;
                result_src = 1'b1;
            end
            OP_SW: if (funct3 == 3'b010) begin
                reg_write  = 1'b0;
                imm_src    = 2'b01;   // S-type
                alu_src    = 1'b1;
                mem_write  = 1'b1;
                result_src = 1'b0;
            end
            OP_R: if (funct7 == 7'b0000000) begin
                reg_write  = 1'b1;
                alu_src    = 1'b0;    // rs2
                result_src = 1'b0;    // ALU result
                alu_op     = 2'b10;
            end
            OP_B: if (funct3 == 3'b000) begin
                imm_src    = 2'b10;   // B-type
                alu_src    = 1'b0;    // compare rs1 with rs2
                branch     = 1'b1;
            end
```

```
60              default: ;
61          endcase
62      end
63
64      // alu_op 00: ADD; 10: R-type selected by funct3; beq: SUB.
65      always @(*) begin
66          case (alu_op)
67              2'b00:   alu_control = (op == OP_B && funct3 == 3'b000) ? 3'b001 : 3'b000;
68              2'b10:   case (funct3)
69                  3'b000:  alu_control = 3'b000;   // add
70                  3'b111:  alu_control = 3'b010;   // and
71                  3'b110:  alu_control = 3'b011;   // or
72                  default: alu_control = 3'b000;
73              endcase
74              default: alu_control = 3'b000;
75          endcase
76      end
77
78  endmodule
79  `endif
```

4.3 Register File

The `regfile` module declares 32 registers of 32 bits each. It provides two asynchronous read ports and one synchronous write port. Reads from address zero return zero, and writes targeting address zero are ignored. The synchronous reset clears the register array.

Listing 3 shows the register file with protected x0, asynchronous reads, synchronous writes, and reset clearing.

Listing 3: Register file (regfile.v)

```
1  `ifndef REGFILE_V
2  `define REGFILE_V
3  `timescale 1ns/1ps
4
5  module regfile (
6      input  wire        clk,
7      input  wire        rst,
8      input  wire [4:0]  address1,      // rs1
9      input  wire [4:0]  address2,      // rs2
10     input  wire [4:0]  address3,      // rd
11     input  wire        write_enable,
12     input  wire [31:0] write_data,
13     output wire [31:0] read_data1,
14     output wire [31:0] read_data2
15 );
16
17     reg [31:0] registers [0:31];
18     integer i;
19
20     always @(posedge clk) begin
21         if (rst) begin
22             for (i = 0; i < 32; i = i + 1)
23                 registers[i] <= 32'b0;
24         end else if (write_enable && (address3 != 5'b0)) begin
25             registers[address3] <= write_data;
26         end
27     end
28
29     assign read_data1 = (address1 == 5'b0) ? 32'b0 : registers[address1];
30     assign read_data2 = (address2 == 5'b0) ? 32'b0 : registers[address2];
31
32 endmodule
33 `endif
```

4.4 Arithmetic Logic Unit

The `alu` module implements ADD, SUB, bitwise AND, and bitwise OR. The zero output is asserted exactly when the 32-bit ALU result is zero. The `beq` instruction uses SUB and this zero output to decide

whether to take the branch. An unrecognized ALU-control value produces a zero result.

Listing 4 gives the combinational ALU operations and zero-flag generation.

Listing 4: Arithmetic logic unit (alu.v)

```
`ifndef ALU_V
`define ALU_V
`timescale 1ns/1ps

module alu (
    input  wire [31:0] src1,
    input  wire [31:0] src2,
    input  wire [2:0]  alu_control,
    output reg  [31:0] alu_result,
    output wire        zero
);

    always @(*) begin
        case (alu_control)
            3'b000:  alu_result = src1 + src2;
            3'b001:  alu_result = src1 - src2;
            3'b010:  alu_result = src1 & src2;
            3'b011:  alu_result = src1 | src2;
            default: alu_result = 32'b0;
        endcase
    end

    assign zero = (alu_result == 32'b0);

endmodule
`endif
```

4.5 Testbench Design

The testbenches use deterministic input vectors and compare outputs or internal state against explicit expected values. Their event controls, clocking, and comparison constructs follow the Verilog simulation model and common RTL verification practices [6, 7]. Sequential core testbenches generate a clock with a 10 ns period, hold reset active for two falling edges, deassert reset, and then wait for a fixed number of clock cycles before checking architectural state. The data-memory testbench checks both asynchronous reads and writes sampled at a rising clock edge. The ALU, immediate-generator, control-unit, and register-file testbenches exercise module behaviour directly. Several testbenches also create Value Change Dump (VCD) files under the `build/` directory when run with Icarus Verilog [8].

5 SIMULATION, VERIFICATION AND RESULTS

5.1 Experimental Setup

Table V describes the environment identified by the supplied project files. The project README states that the RTL was validated with Icarus Verilog 11.0; the Makefile defines the reproducible test workflow. [2, 8, 9]

TABLE V. SIMULATION ENVIRONMENT

Item	Details
HDL	Verilog HDL
Simulator	Icarus Verilog 11.0
Waveform viewer	GTKWave
Instruction memory	64 words
Data memory	64 words
Core-test clock	10 ns period (5 ns half-period delay)
Verification strategy	Module-level testbenches and full-core integration tests

5.2 Module-Level Verification

The module-level testbenches verify the functions summarized in Table VI. All six module-level targets are successfully passed.

TABLE VI. MODULE-LEVEL TEST COVERAGE AND RESULTS

Testbench	Checks defined in the testbench	Result
`regfile_test`	Register writes and reads, simultaneous read ports, x0 protection, disabled writes, and reset clearing selected registers	PASS
`instruction_memory_test`	Instruction lookup at selected byte addresses, empty memory location, and ignored low address bits	PASS
`data_memory_test`	Initialized reads, synchronous writes, asynchronous read-back, and disabled-write behaviour	PASS
`signext_test`	Five I-type and two S-type immediate extraction/sign-extension assertions	PASS
`alu_test`	Addition, subtraction, wraparound, zero flag, AND, OR, and default ALU-control behaviour	PASS
`control_test`	Load/store controls, ADD/AND/OR decode, supported opcode defaults, and SUB encoding	PASS

Table VII records the seven expected comparisons encoded directly in the archived `signext_test.v`.

TABLE VII. IMMEDIATE-GENERATOR EXPECTED RESULTS FROM THE ARCHIVED TESTBENCH

Test vector	Expected output (hex)
I-type −4	`FFFFFFFC`
I-type +8	`00000008`
I-type +2047	`000007FF`
I-type −2048	`FFFFF800`
I-type 0	`00000000`
S-type +12	`0000000C`
S-type −4	`FFFFFFFC`

5.3 Core-Level Test Cases

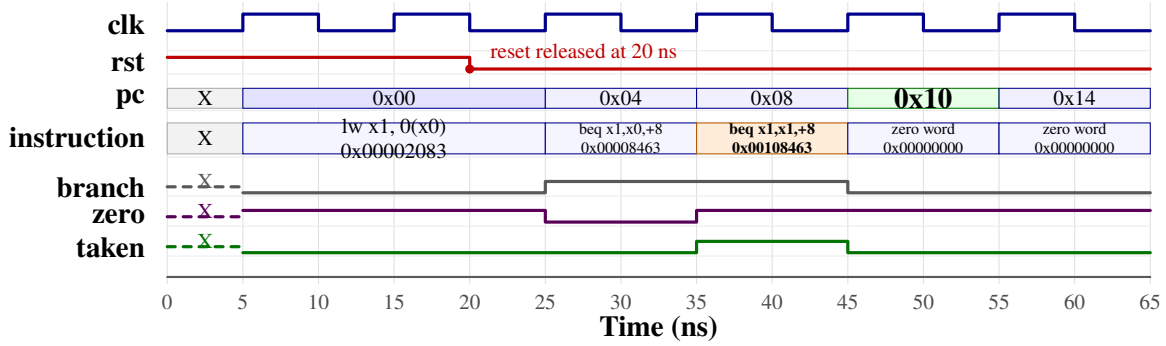
Table VIII records key checks that are explicitly present in the integration testbenches.

TABLE VIII. CORE-LEVEL TEST CASES AND ASSERTIONS

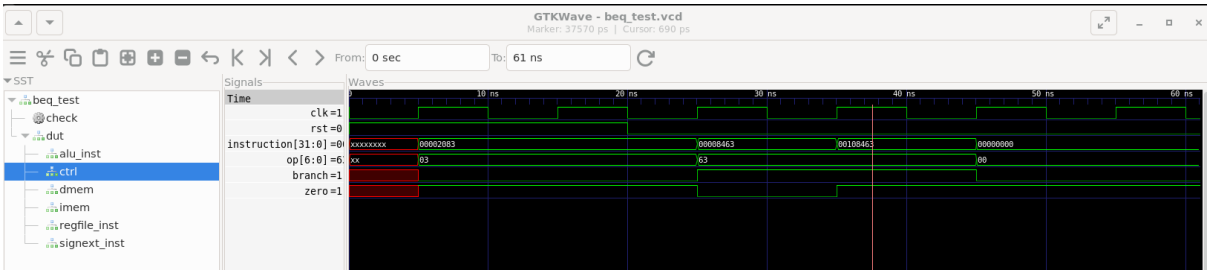
Testbench	Key checks encoded in testbench	Status
`main_test`	After five load instructions: x9=`0x00000010`, x6=`0xCAFEF00D`, x7=`0xDEADBEEF`, x0=0, x10=`0x12345678`, PC=`0x14`.	PASS
`sw_test`	Checks stores to data-memory words 7, 4, and 5 with expected values `0xAAAAAAAA`, `0xCAFEF00D`, and `0x12345678`; also checks x0 and PC=`0x1C`.	PASS
`r_type_test`	Checks x7=`0x75A99AB7` (ADD), x8=`0x8AAAA008` (AND), x9=`0xEAFEFAAF` (OR), x10=`0x60A89AA7`, x0=0, and PC=`0x1C`.	PASS
`beq_test`	Checks x1=5, x2=0 because the taken branch skips its load, and PC=`0x14` after the tested branch sequence.	PASS

5.4 Simulation Waveform

All timing sequences were verified using GTKWave. For example, the BEQ timing sequence was verified in GTKWave using the generated `beq_test` VCD trace. The branch at PC 0x04 is not taken, while the branch at PC 0x08 is taken and redirects execution to 0x10, skipping the load at 0x0C. Final checks confirm x1=5, x2=0, and PC=0x14.



(a) Expected timing diagram of the BEQ integration test.



(b) GTKWave trace of `beq_test.vcd` (simulation output).

Fig. 2. Timing diagram of the BEQ integration test (a), checked against the GTKWave trace (b).

5.5 Results and Discussion

All ten Makefile targets passed: six module-level tests and four integration tests. The test suite covers registers, memories, immediate extraction, ALU/control logic, loads, stores, R-type operations, and

conditional branching. The BEQ simulation waveform was inspected in GTKWave. Synthesis, FPGA, timing-closure, power, and hardware-measurement results are not claimed.

In the archived RTL, `signext.raw_src` is declared as `[31:7]` and connected to `instruction[31:7]`; therefore `raw_src[7]` denotes instruction bit 7, the B-type immediate bit 11. This indexing is internally consistent.

6 INDIVIDUAL CONTRIBUTION

Table IX records each member's assigned implementation responsibility and the corresponding source files and testbench evidence.

TABLE IX. INDIVIDUAL CONTRIBUTION

Team member	Section / ID	Individual tasks performed	Evidence
Kuldeep Bhurani	K/251090052182	Implemented the `lw` effective-address and memory-data write-back path.	`control.v`, `lw_test`, `main.v`, `main_test`, `regfile.v`
Saksham Kanodia	K/251090052618	Implemented the `sw` effective-address, store-data routing, and memory-write path.	`control.v`, `instruction_memory.v` ,`main.v`, `sw_test`
Yashasvi Sharma	K/251090052336	Implemented ADD and OR ALU/control operations.	`alu.v`, `control.v`, `main.v`, `r_type_test`
Amarnath K.	K/251090052394	Implemented the `beq` comparison, branch decision, and next-PC update.	`beq_test`, `control.v`, `data_memory.v`, `main.v`

6.1 Contribution Statement

We divided instruction implementation across four members: Kuldeep Bhurani implemented `lw`; Saksham Kanodia implemented `sw`; Yashasvi Sharma implemented ADD and OR; and Amarnath K. implemented `beq`. These components were integrated into the processor and evaluated using the supplied module-level and core-level Icarus Verilog regression tests. Table IX links each responsibility to the relevant RTL files and testbench.

7 CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The implemented 32-bit single-cycle core supports `lw`, `sw`, `beq`, `add`, `and`, and `or`. All ten Makefile targets passed. The archived `signext_test` contains seven expected comparisons: five I-type and two S-type vectors; B-type immediates are exercised by the BEQ integration test, but have no separate boundary-vector test.

7.2 SDG 9: Industry, Innovation and Infrastructure

UN Sustainable Development Goal 9 calls for resilient infrastructure, sustainable industrialization, and innovation. This project relates most directly to Targets 9.5 and 9.b, which emphasize research capacity and technology development [11]. The open RISC-V instruction-set architecture and modifiable Verilog RTL make processor design available for inspection, experimentation, and reuse without relying on a proprietary instruction set [1]. Separate datapath modules and repeatable testbenches let students examine instruction execution, debug control and datapath behaviour, and validate changes through simulation. This modularity also makes experiments easier to reproduce and lets students compare design choices against a documented ISA, supporting technical learning and familiarity with open engineering practices. These are limited but relevant educational contributions to innovation and capability-building. The alignment is indirect: this is a simulation-tested educational processor, not a fabricated chip or industrial deployment, and no direct infrastructure or economic impact has been measured. Future work could strengthen the connection through synthesis and FPGA evaluation, comparative resource measurements, or reuse of the core in a broader teaching or research setting.

7.3 Limitations and Future Scope

The processor provides a foundation for investigating the trade-offs between architectural simplicity, execution latency, and hardware efficiency. Future research could explore synthesis-driven optimization of critical paths, logic sharing, and control-signal generation to improve power, performance, and area (PPA). Formal verification and constrained-random testing could strengthen confidence in functional correctness across instruction sequences and corner cases.

REFERENCES

[1] RISC-V International, "The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture," ratified version 20260120, Jan. 2026. [Online]. Available: `https://docs.riscv.org/reference/isa/unpriv/`. [Accessed: Oct. 21, 2026].

[2] "32bitcpu" project, *README.md*, RTL source, testbenches, hex test files, and Makefile. Available: `https://github.com/Kuldeep-Bhurani/32bitCPU/`

[3] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*, 2nd ed. Cambridge, MA, USA: Morgan Kaufmann, 2020.

[4] S. L. Harris and D. Harris, *Digital Design and Computer Architecture, RISC-V Edition*. Cambridge, MA, USA: Morgan Kaufmann, 2021.

[5] RISC-V Non-ISA Specifications, "RISC-V Assembly Programmer's Manual." [Online]. Available: `https://github.com/riscv-non-isa/riscv-asm-manual`. [Accessed: Oct. 21, 2026].

[6] IEEE, *IEEE Standard for Verilog Hardware Description Language*, IEEE Std. 1364-2005, Apr. 2006, doi: 10.1109/IEEESTD.2006.99495.

[7] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2003.

[8] Icarus Verilog, "Icarus Verilog Documentation." [Online]. Available: `https://steveicarus.github.io/iverilog/`. [Accessed: Oct. 21, 2026].

[9] Free Software Foundation, "GNU Make Manual," ver. 4.4.1, Feb. 2023. [Online]. Available: `https://www.gnu.org/software/make/manual/make.html`. [Accessed: Oct. 21, 2026].

[10] GTKWave Project, "GTKWave Documentation: Waveform Viewer for Verilog VCD and Other Trace Formats." [Online]. Available: `https://gtkwave.github.io/gtkwave/`. [Accessed: Oct. 21, 2026].

[11] United Nations, "Goal 9: Industry, Innovation and Infrastructure." [Online]. Available: `https://www.un.org/sustainabledevelopment/infrastructure-industrialization/`. [Accessed: Oct. 21, 2026].